

Course

« *Computer Vision* »

Sid-Ahmed Berrani

2025-2026



XII. The SIFT Descriptor

The SIFT descriptor

Each **detected keypoint** is characterized by:

1. A coordinate in the image space (x,y)
2. A scale
3. An orientation

Next step:

We have to compute a signature/descriptor/feature vector for each keypoint.

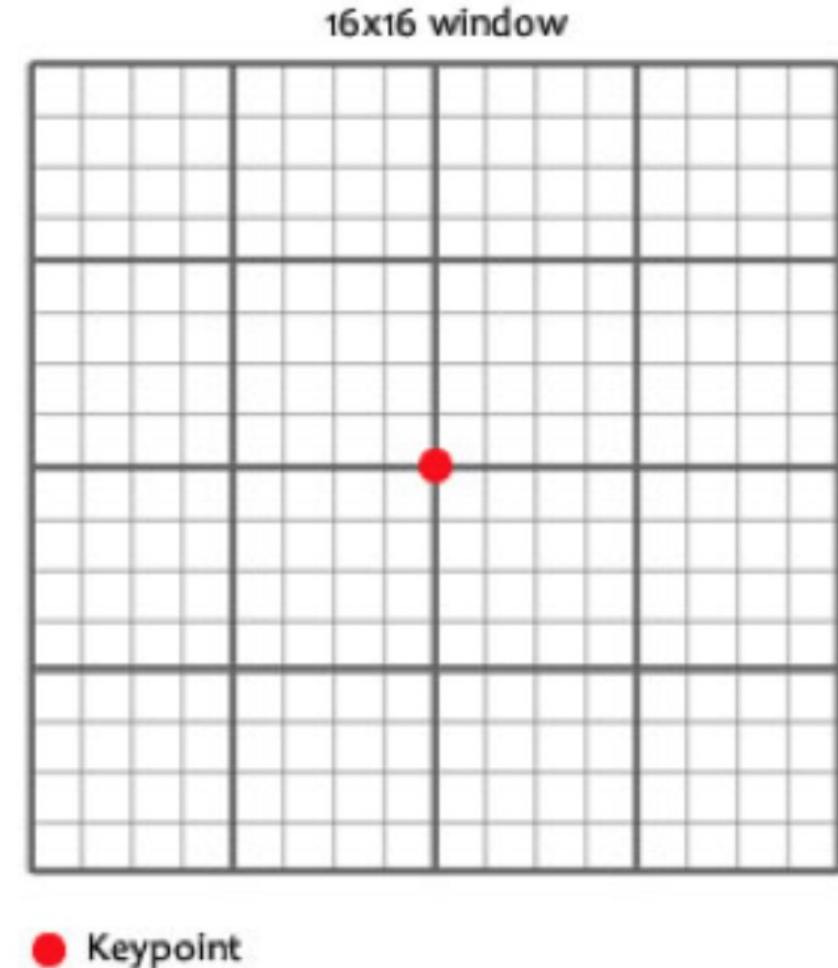
It aims to uniquely identifying the ketpoint with respect to the others.

XII. The SIFT Descriptor

The SIFT descriptor

- To create the descriptor, we consider gradient vectors in a 16x16 window around each keypoint, at the keypoint's scale.
 - The size of this region is adapted based on the keypoint's scale (e.g., scaled relative to σ).
 - Here, the window is rotated with respect to the keypoint orientation.
- ⇒ A descriptor that is **invariant to scale** and **orientation**.

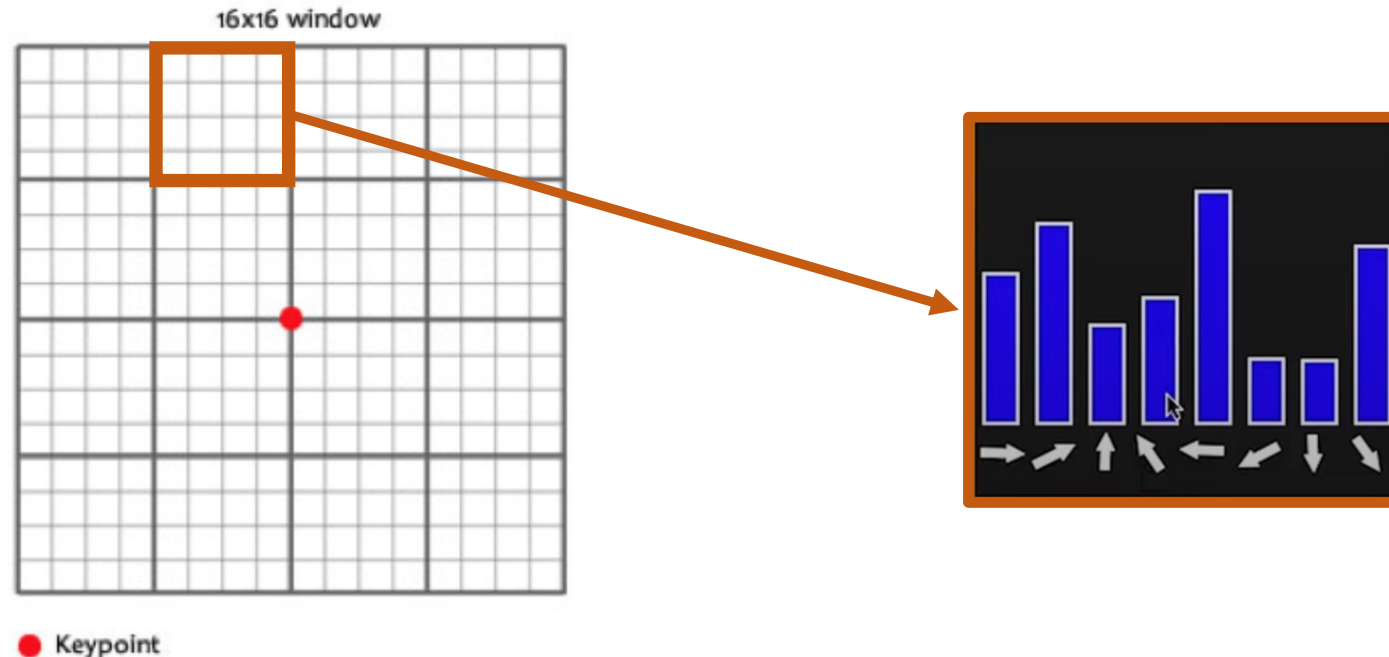
The descriptor is computed in the Gaussian-blurred image corresponding to the keypoint's scale and octave, not in the DoG or original image.



XII. The SIFT Descriptor

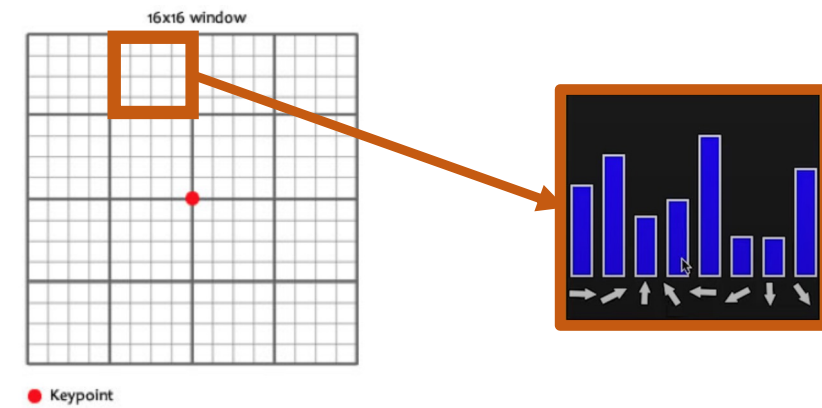
The SIFT descriptor

- The 16x16 window is divided into 16 4x4 cells. This provides **spatial granularity**, capturing gradient distributions in different parts of the neighborhood.
- For each 4x4 cell, an 8-bins (45° steps: 0°, 45°, 90°, ..., 315°) histogram of gradient orientations is formed.
- The gradient is computed at each pixel, both in terms of magnitude and orientation, but we focus on the orientation.



XII. The SIFT Descriptor

The SIFT descriptor



Let's focus on how orientation histograms are built.

- For each pixel in the 16×16 cell, compute gradient magnitude and orientation.
- Optionally, apply a **Gaussian weighting** (centered at the keypoint) to give more importance to gradients near the center.
- Each pixel inside the cell votes for a bin:

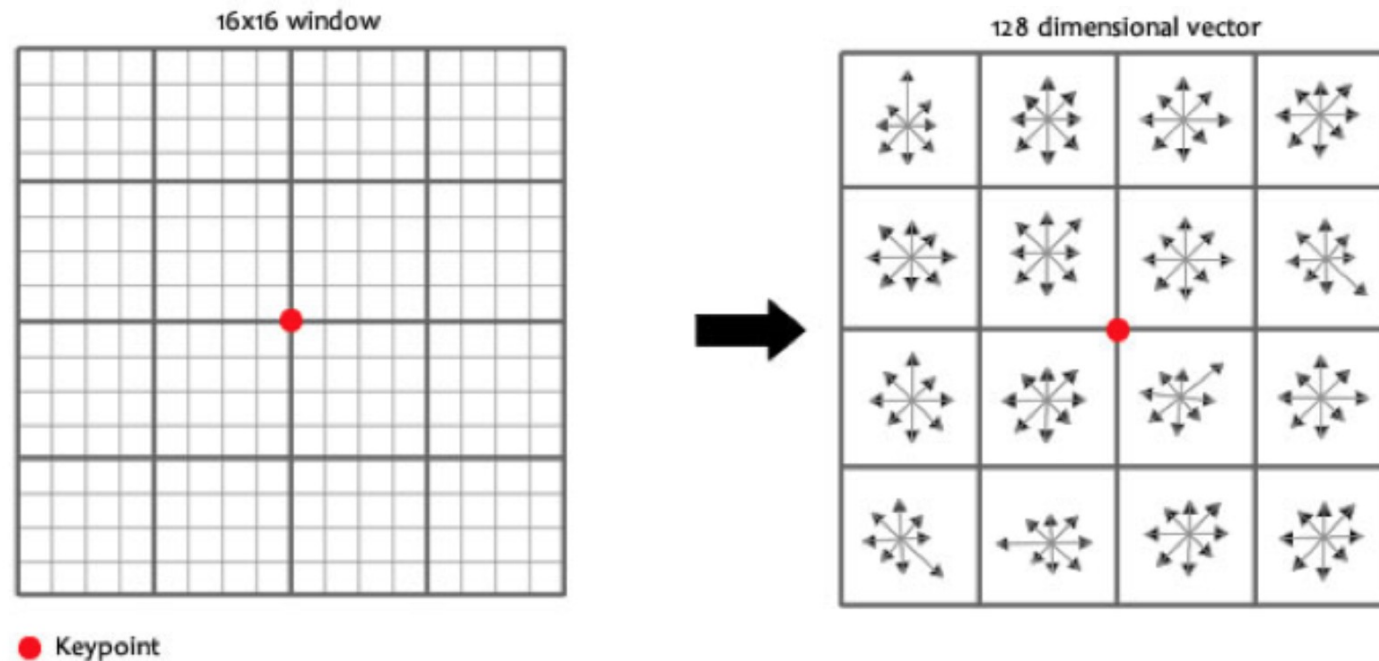
Vote weight = gradient magnitude × Gaussian weight.

Orientation is relative to the keypoint's orientation → **rotation invariance**

XII. The SIFT Descriptor

The SIFT descriptor

- Histograms computed for cells are concatenated all together to produce a $16 \times 8 = 128$ -values feature vector (a 128-dimensional descriptor/signature).



XII. The SIFT Descriptor

The SIFT descriptor

Descriptor normalization

- The resulting 128-dimensional vector is normalized to unit length (L2 norm):

$$\text{descriptor} \leftarrow \frac{\text{descriptor}}{\|\text{descriptor}\|_2}$$

where $\|\cdot\|_2$ is the Euclidean norm.

Why Normalize?

To achieve **illumination invariance**.

- Brightness changes scale all gradient magnitudes.
- After normalization, only the **relative distribution of gradient strengths** matters, not their absolute magnitudes.

XII. The SIFT Descriptor

The SIFT descriptor

Descriptor normalization – Clipping and Renormalization

- To further reduce sensitivity to illumination changes:
 - Clamp large values: values > 0.2 are set to 0.2.
 - Normalize again
- => Prevents a few large gradients (e.g., edges) from dominating the descriptor.
- => Encourages more balanced and distinctive descriptors.

XII. The SIFT Descriptor

Descriptor matching

How can we compare to SIFT descriptors?

Let H_1 and H_2 two descriptors of dimension 128.

- The **Euclidean distance** can be used to measure the similarity between two descriptors:

$$d(H_1, H_2) = \sqrt{\sum_k (H_1(k) - H_2(k))^2}$$

- **Histogram intersection:**

$$d(H_1, H_2) = \sum_k \min(H_1(k), H_2(k))$$

Larger the intersection distance, better the match.

XIII. Other Image Descriptors

Many other image descriptors have been proposed

- **Local Binary Pattern** (LBP) descriptor is a powerful texture descriptor that is conceptually much simpler and faster than SIFT, though with some trade-offs in robustness and descriptiveness.
- It captures local texture by encoding the local spatial structure of an image, specifically, how the intensity of a center pixel compares to its neighbors.
- **Idea:**
 - For each pixel:
 - Take a neighborhood (usually 3×3).
 - Compare each neighbor to the center pixel:
 - If neighbor $\geq$ center $\rightarrow$ write 1
 - Else $\rightarrow$ write 0
 - This gives an 8-bit binary number (for 3×3).

XIII. Other Image Descriptors

Many image descriptors have been proposed

- **LBP Histogram**

Over a region (e.g., 16×16), compute the **histogram of LBP values**.

The histogram is the **descriptor** for that region.

It captures the **texture pattern** (e.g., edges, corners, flat areas).

- **Variants :**

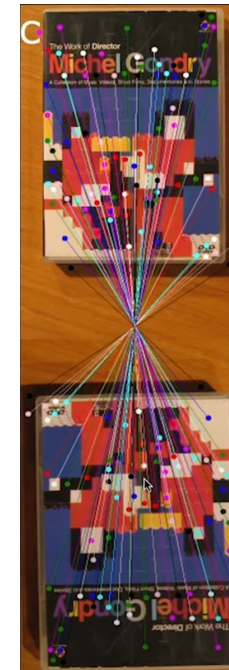
Rotation-invariant LBP: Encodes binary patterns that remain constant under rotation.

Multi-scale LBP: Compares neighbors at larger radii.

XIV. Applications of Image Feature Matching

Image matching

- Feature matching refers to the process of detecting and describing interest points (keypoints) in images and then finding correspondences between them across different images.
- This is foundational in many computer vision tasks.



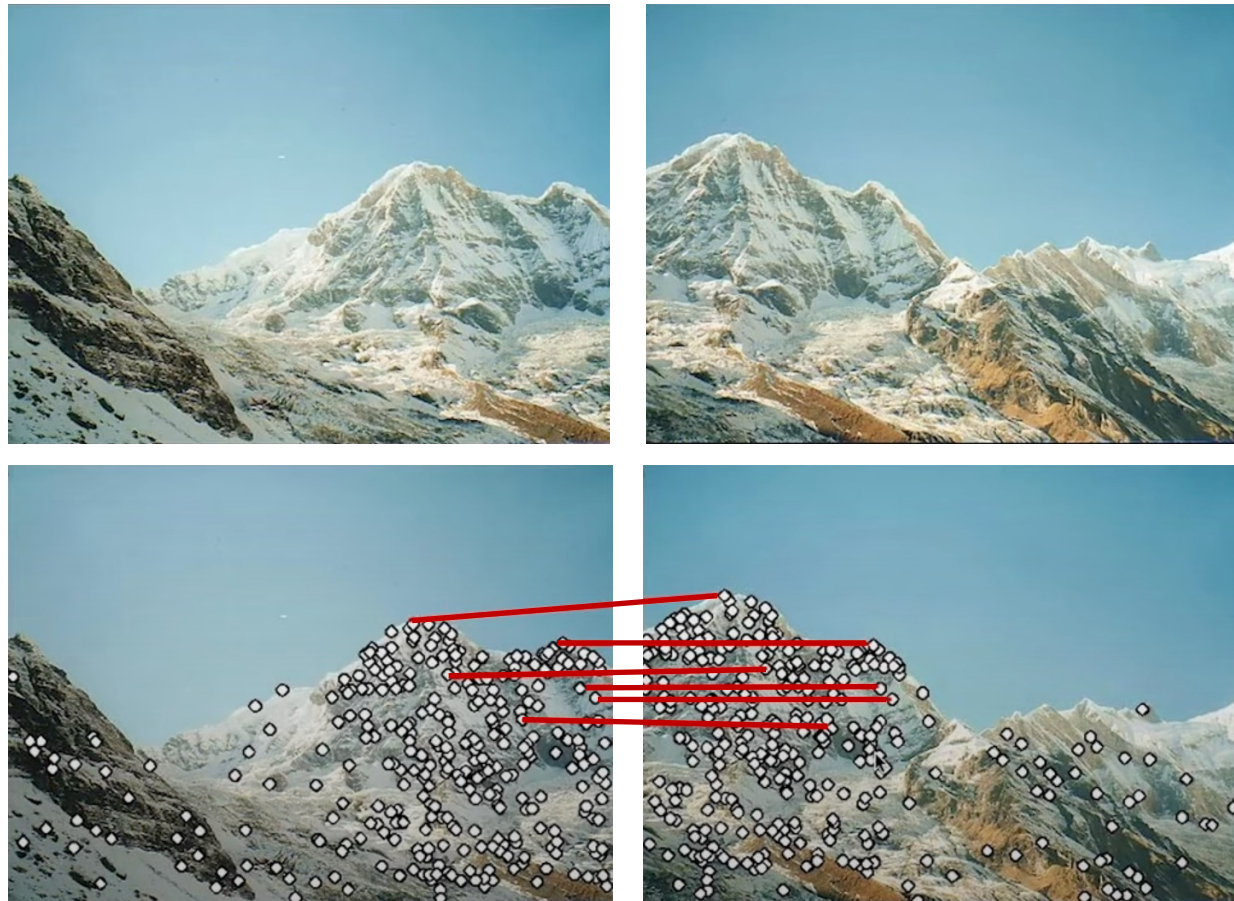
XIV. Applications of Image Feature Matching

1. Image Stitching / Panorama Creation

- **Goal:** Combine overlapping photos into one seamless panorama.
- **How SIFT helps:**
 - Match features across overlapping regions.
 - Estimate homography between images.
 - Warp and blend the images.

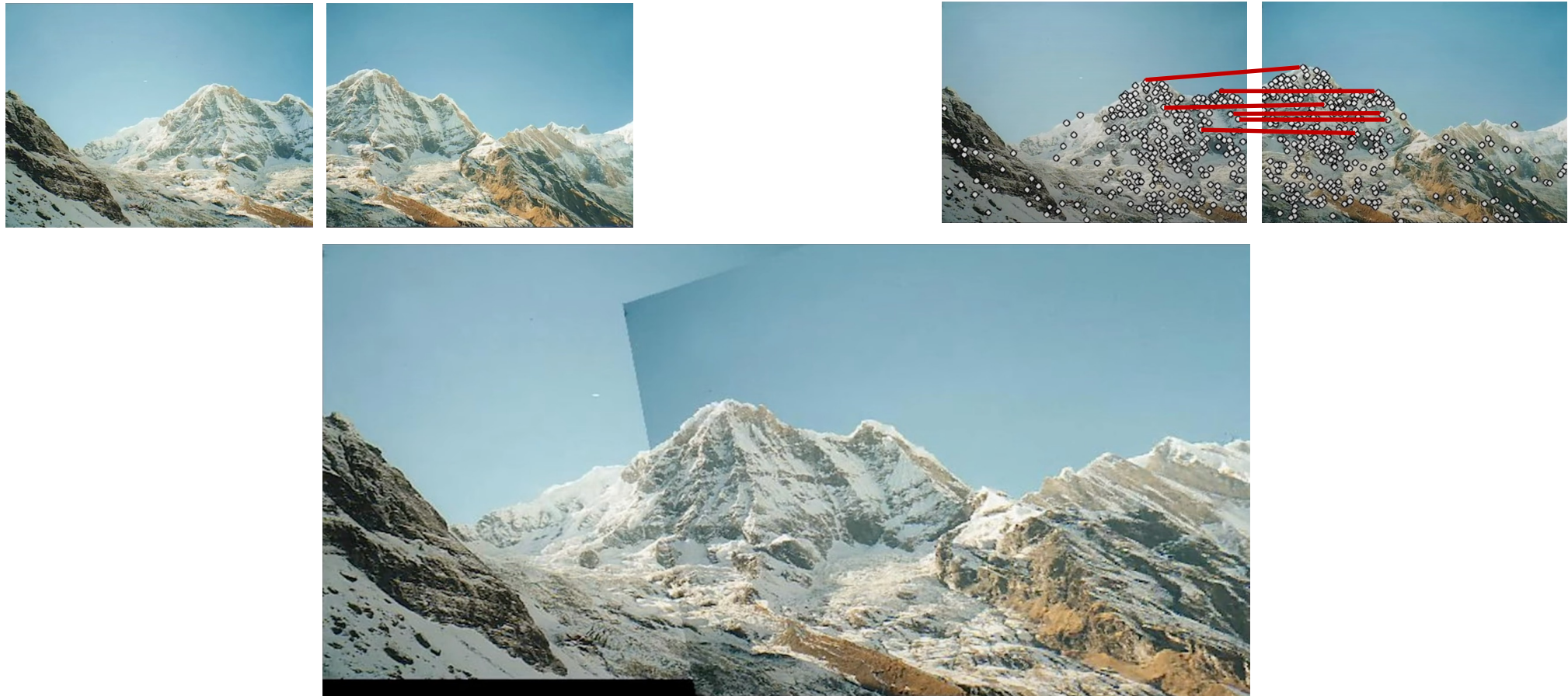
XIV. Applications of Image Feature Matching

1. Image Stitching / Panorama Creation



XIV. Applications of Image Feature Matching

1. Image Stitching / Panorama Creation



XIV. Applications of Image Feature Matching

2. Video Stabilization

- **Goal:** Remove jitter from hand-held videos.
- **How SIFT helps:**
 - Track keypoints between frames.
 - Estimate motion model (e.g., affine transform).
 - Smooth the trajectory and warp frames accordingly.

XIV. Applications of Image Feature Matching

3. Object Recognition / Instance Recognition

- **Goal:** Recognize known objects in new scenes or images.
- **How SIFT helps:**
 - Compare keypoints from the query image to a database of objects.
 - Robust to scale, viewpoint, and illumination changes.
- **Example:** Recognizing a specific logo, book cover, or product in an image.

XIV. Applications of Image Feature Matching

5. Image Fingerprinting – Copy-Move Forgery Detection

- **Goal:** Detect parts of an image that were copied and pasted elsewhere in the same image.
- **How SIFT helps:**
 - Match features in different regions.
 - Identify consistent affine transformations among matches.

XIV. Applications of Image Feature Matching

5. Image Fingerprinting – Copy-Move Forgery Detection

The **Orange Labs** solution @ TRECVID 08



XIV. Applications of Image Feature Matching

5. Image Fingerprinting – Copy-Move Forgery Detection

The **Orange Labs** solution @ TRECVID 08



XIV. Applications of Image Feature Matching

6. Image Registration

- **Goal:** Align two or more images of the same scene.
- **How SIFT helps:**
 - Detect and match features in both images.
 - Estimate geometric transform (rigid, affine, or projective).
- **Applications:**
 - Medical image registration (CT vs MRI)
 - Satellite image alignment (multi-temporal or multi-spectral)

XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Goal: Retrieve visually similar images from a database given a query image.

How SIFT helps:

- Extract and match local descriptors from the query image and database images.
- Use techniques like **Bag-of-Words (BoW)** or **VLAD** (Vector of Locally Aggregated Descriptors).
- Enables **scale- and rotation-invariant** similarity search, even in cluttered scenes.

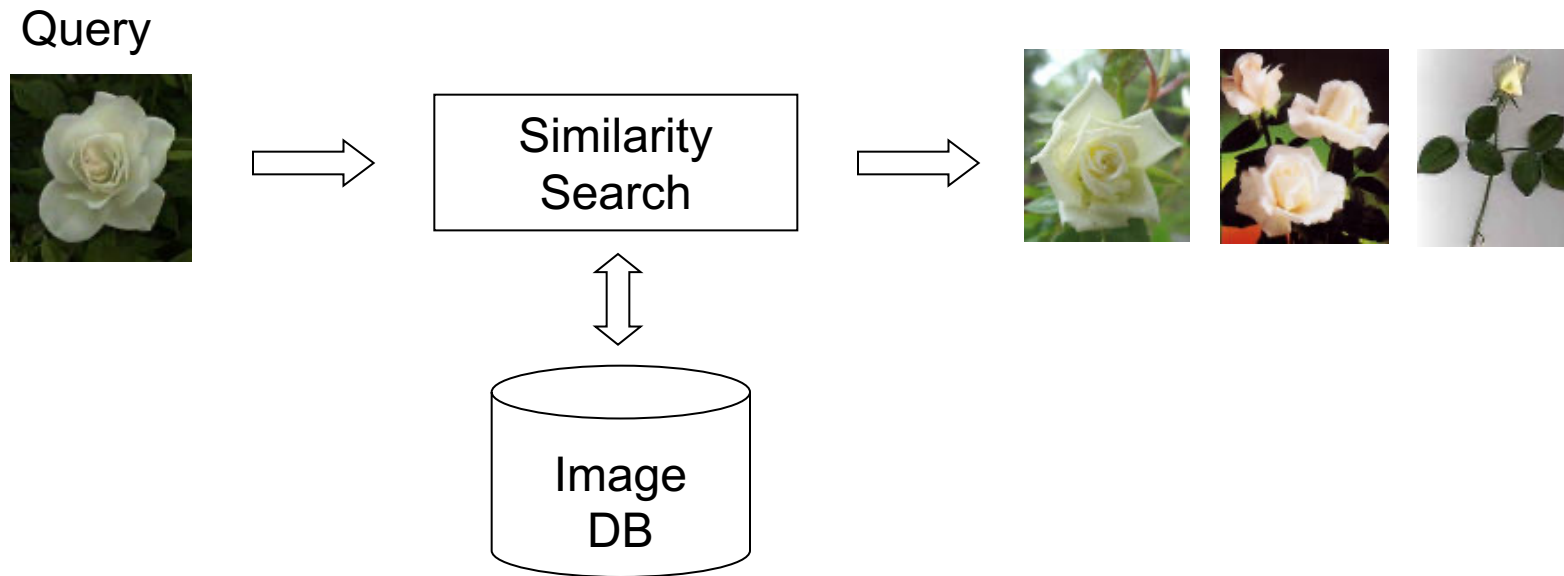
Applications:

- Reverse image search
- Product search in e-commerce
- Medical image retrieval
- Museum/artwork archives

XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

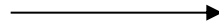
Query by example



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Query by example



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Query by example for content enrichment



Salon d'Hercule : Le Repas chez Simon le Pharisien. 1576. Commandée par les pères servites pour leur réfectoire à Venise, fut offerte à Louis XIV par la République de Venise en 1664...

XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

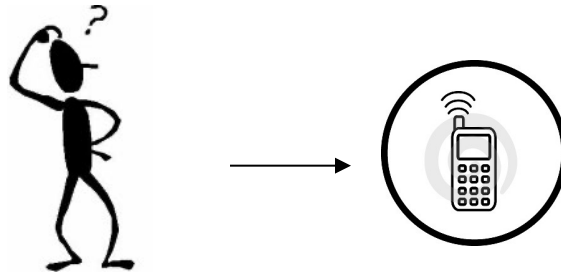
Query by example for content enrichment



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Query by example for mapping applications



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Query by example for mobile visual search



XIV. Applications of Image Feature Matching

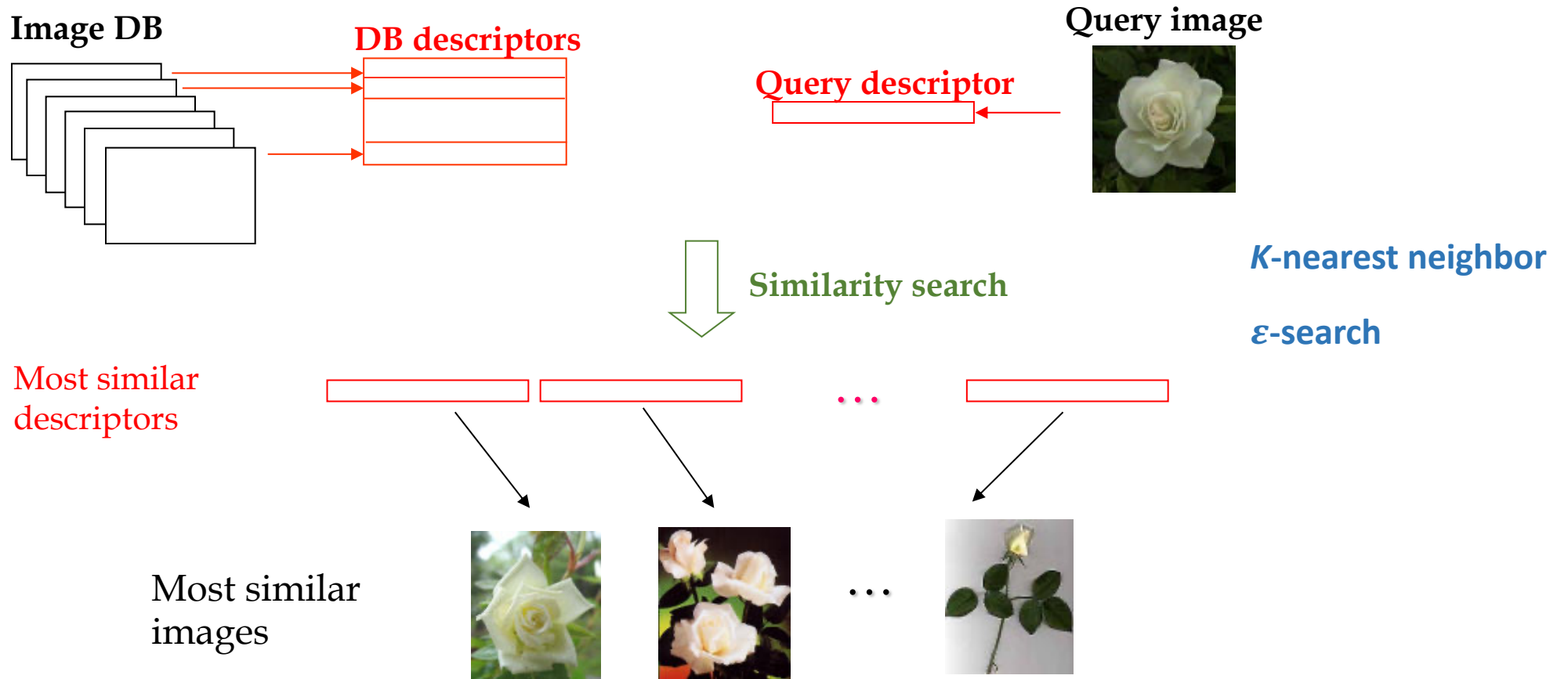
4. Content-Based Image Retrieval / Similarity Search

Query by example for mobile visual search



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

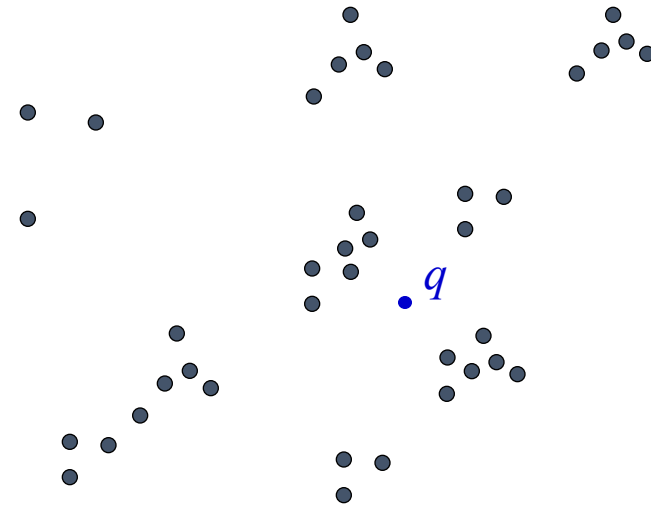


XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Nearest-Neighbor Searches

- Sequential Scan: Trivial but inefficient

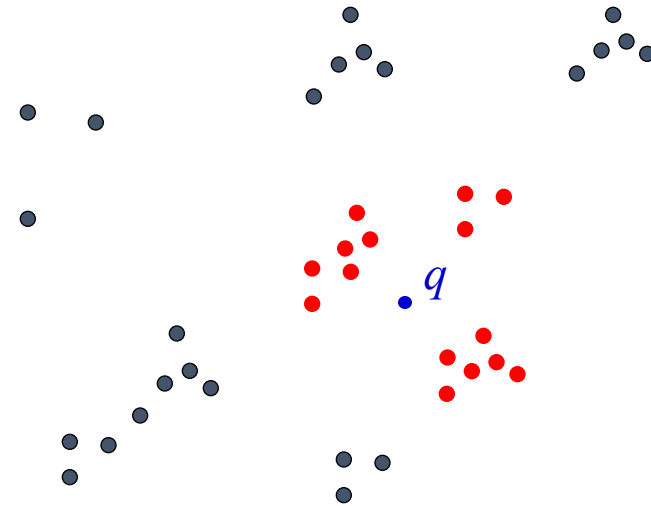


XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Nearest-Neighbor Searches

- Sequential Scan: Trivial but inefficient
- Multidimensional Indexing:
 - Prune the search space

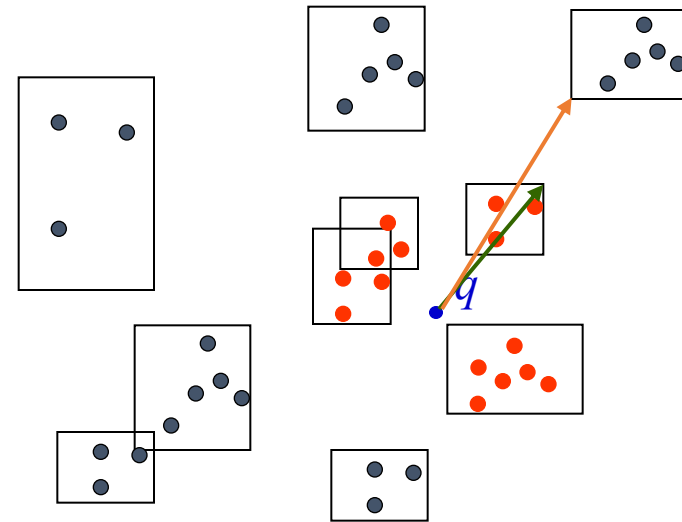


XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

Nearest-Neighbor Searches

- Sequential Scan: Trivial but inefficient
- Multidimensional Indexing:
 - Prune the search space
 - Group vectors into cells
 - Use geometrical rules to filter irrelevant cells
 - R-Tree, SS-Tree, SR-Tree,...



XIV. Applications of Image Feature Matching

4. Content-Based Image Retrieval / Similarity Search

The nearest neighbor search with local image features (like SIFT)

Query image $\Rightarrow$ A set of query descriptors: $\{Q_1, Q_2, \dots, Q_n\}$

k-nearest neighbor search

Q_1	Img1	Img3	...	Img9
Q_2	Img2	Img1	...	Img10
			$\vdots$	
Q_n	Img1	Img3	...	Img5



Img Id	#votes
Img1	123
Img2	110
Img3	93
Img4	5
Img5	5
Img6	4
Img7	4
$\vdots$	$\vdots$

Page-Hinkley
Test Decision

XV. Bag of Visual Words

